PIPELINEIQ // SELF-HEALING MLOPS PLATFORM

Comprehensive User Manual & System Architecture Guide

B.Tech Thesis Project Codebase & Operational Manual

Submitted by: Group No. 10

- Maimoona Manzoor (CSE-22-LE-70)
- Syed Maryam Andrabi (CSE-22-LE-74)
- Momin Zahoor (CSE-22-LE-71)

Supervised by: **Dr. Sahil Sholla** (Assistant Professor, Dept. of CSE)

Institution: Islamic University of Science and Technology (IUST), Awantipora

Academic Session: Spring 2026

Executive Abstract: PipelineIQ is an enterprise-grade cloud-native MLOps platform engineered to fully automate data ingestion, minority balancing via SMOTE, multi-model concurrent execution (Random Forest, XGBoost, LightGBM), MLflow experiment tracking, statistical covariate drift detection (Evidently AI, KS-Test, PSI), and zero-downtime self-healing model retraining.

1. Executive Summary & Rationale

According to empirical findings by Sculley et al. (*NeurIPS 2015*), up to 90% of technical debt in deployed machine learning systems stems from operational infrastructure—data validation, feature scaling, model drift monitoring, and manual retraining workflows—rather than the underlying model mathematics. PipelineIQ directly addresses this challenge by automating the 70% manual engineering overhead required to maintain production machine learning models.

Key Objectives Fulfilled:

- **Automated Preprocessing & SMOTE Balancing:** Implements missing value median/mode imputation, quantile normalization, and Synthetic Minority Over-sampling Technique (SMOTE).
- **Concurrent Multi-Model Execution:** Simultaneously trains Random Forest, XGBoost, and LightGBM models with hyperparameter depth tuning.
- **Statistical Drift Telemetry:** Continuously evaluates Population Stability Index (PSI) and 2-sample Kolmogorov-Smirnov statistics.
- **Autonomous Self-Healing:** Dynamically dispatches background retraining threads upon drift threshold breach (PSI > 0.25) to hot-swap active champion models.

2. Four-Layer System Methodology

Layer	Core Components	Key Responsibilities
Layer 1: Ingestion & SMOTE	FastAPI, Pandas, Imbalanced-Learn	Multipart CSV ingestion, target detection, missing value median imputation, SMOTE balancing.
Layer 2: AutoML Execution	Scikit-Learn, XGBoost, LightGBM, MLflow	Concurrent multi-model training, decision threshold calibration, MLflow experiment tracking.
Layer 3: Serving & Drift	FastAPI, JWT, SciPy, Evidently AI	JWT Bearer security, 24h tokens, 2-sample KS-Test & PSI drift calculations, self-healing trigger.
Layer 4: Orchestration	Docker, Docker Compose, React Vite	Containerized multi-service deployment (React UI port 3000, FastAPI port 8000, MLflow port 5001).

3. Installation & Quick-Start Execution Guide

Option A: Instant Demo Simulation Mode (Zero Backend Setup Needed)

Ideal for quick evaluation or committee presentation on laptops without Python/MongoDB setup:

```
# 1. Install Node modules
cd June/June
npm install

# 2. Launch Vite development server
npm run dev

# 3. Open browser at http://localhost:3000
```

Option B: Production Docker Compose Execution

Launches full stack (FastAPI Backend, React Web UI, MongoDB, MLflow Registry):

```
cd June/June
docker-compose up --build -d

# Access points:
# React UI: http://localhost:3000 | FastAPI Docs: http://localhost:8000/docs | MLflow: http://lo
calhost:5001
```

4. Step-by-Step Operator User Manual

Step 1: User Authentication & JWT Handshake

- Navigate to http://localhost:3000.
- Enter pre-filled operator credentials (admin@pipelineiq.io / admin123) or click Register.
- Upon submission, FastAPI validates password hash via bcrypt and returns an encrypted JWT Bearer access token stored in Zustand client memory.

Step 2: Tabular Dataset Upload & Pipeline Dispatch

- On the Dashboard Overview, click '+ New Pipeline Setup'.
- Drag and drop any tabular dataset CSV (e.g., ibm_hr_attrition.csv) into the dropzone container.
- Select custom models to execute concurrently: [x] Random Forest Engine, [x] XGBoost Engine, [x] LightGBM Engine.
- Choose target evaluation metric (F1-Score, Accuracy, or ROC-AUC).
- Click 'Launch Training Pipeline' to trigger SMOTE preprocessing and background worker execution.

Step 3: Real-Time Telemetry & Champion Model Promotion

- The dashboard updates live every 5000ms using TanStack Query polling.
- A temporary yellow 'Training...' placeholder badge provides immediate zero-latency feedback.
- Upon completion, metrics are evaluated, and the top-performing algorithm is automatically awarded a green 'CHAMPION MODEL' badge.

Step 4: Statistical Data Drift Monitoring

- Inspect the Evidently AI Drift Monitor panel at the bottom of the dashboard.
- Under baseline operation, the telemetry monitor displays a green STABLE status with PSI < 0.10 and KS-Test p-value > 0.05.

Step 5: Injecting Synthetic Covariate Shift & Alert State

- Click 'Inject Covariate Shift' to simulate real-world data corruption/behavioral shift.
- The statistical engine calculates PSI > 0.25 (e.g. PSI = 0.38), triggering a high-visibility yellow alert border.
- The banner turns blue/yellow: 'CRITICAL DATA DRIFT DETECTED — AUTONOMOUS RETRAINING IN PROGRESS'.

Step 6: Autonomous Self-Healing Recovery

- Algorithm 4 dispatches background worker thread to tune hyperparameter depth and retrain candidate models on shifted data.
- Once retrained, the system hot-swaps the champion model registry.
- Click 'Reset Drift Baseline' to return system telemetry to STABLE HEALTHY status.

Operational Note: The 5-second deterministic polling strategy uses refetchIntervalInBackground: true to ensure telemetry updates continue seamlessly even while an operator switches tabs during model training.

5. Mathematical Foundations & System Algorithms

5.1 Synthetic Minority Over-sampling Technique (SMOTE)

To eliminate class imbalance in datasets like employee attrition (84% Stayed vs 16% Left), SMOTE synthesizes minority class samples along feature-space line segments connecting k-nearest neighbors:

$$x_{new} = x_i + \lambda (x_{zi} - x_i), \text{ where } \lambda \sim Uniform(0, 1)$$

5.2 Population Stability Index (PSI)

Population Stability Index quantifies shift between baseline reference data and live production distributions across 10 quantile bins:

$$PSI = \Sigma [(P_{current,b} - P_{baseline,b}) * ln(P_{current,b} / P_{baseline,b})]$$

PSI Range	Statistical Classification	System Action
PSI < 0.10	No Significant Drift	Maintain standard model serving state.
0.10 <= PSI <= 0.25	Moderate Covariate Shift	Flag telemetry warning in audit log.
PSI > 0.25	Critical Covariate Shift	Trigger Algorithm 4 Autonomous Retraining.

5.3 2-Sample Kolmogorov-Smirnov Test

Compares empirical cumulative distribution functions (eCDF) F_1(x) and F_2(x). Drift is signaled when p-value < 0.05:

$$D = \sup_x | F_{reference}(x) - F_{current}(x) |$$

6. REST API Endpoint Reference

Method	Endpoint Path	Auth	Description
POST	/api/v1/auth/register	None	Register new user & MLflow profile.
POST	/api/v1/auth/login	None	Authenticate & issue JWT Bearer token.
GET	/api/v1/health	None	Check FastAPI, MongoDB, MLflow status.
POST	/api/v1/models/train	Bearer	Dispatch multi-model AutoML pipeline.
GET	/api/v1/runs	Bearer	Fetch live MLflow run telemetry history.
GET	/api/v1/drift	Bearer	Get Evidently AI KS & PSI drift telemetry.
POST	/api/v1/drift/inject	Bearer	Inject artificial covariate shift.

POST	/api/v1/drift/reset	Bearer	Reset drift baseline metrics to stable.

7. Troubleshooting & Thesis Acknowledgements

Common Issues & Solutions:

- **Passlib / Bcrypt Module Error:** Run pip install passlib bcrypt==4.0.1 in virtual environment.
- **MongoDB Connection Timeout:** Ensure mongod is running locally on port 27017 or set MONGO_URL environment variable.
- **XGBoost OpenMP Error on macOS:** Install OpenMP library via Homebrew: brew install libomp.

Academic Thesis Acknowledgements:

We express our sincere gratitude to our supervisor, **Dr. Sahil Sholla**, Assistant Professor in the Department of Computer Science and Engineering at the Islamic University of Science and Technology (IUST), Awantipora, Kashmir, for his invaluable guidance, technical insight, and steadfast support throughout the design and execution of this thesis project.